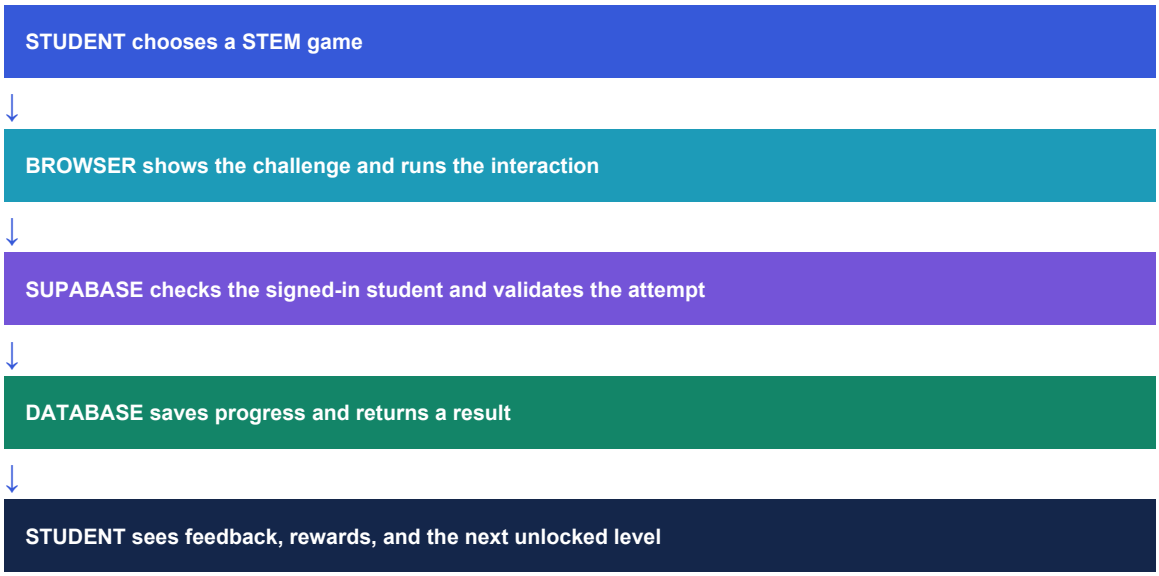
ROBOLOGIC STEM LAB | TECHNICAL FIELD GUIDE

How the whole system works

A beginner-friendly guide to the games, data flow, security, accounts, database, technology stack, and tomorrow’s presentation.



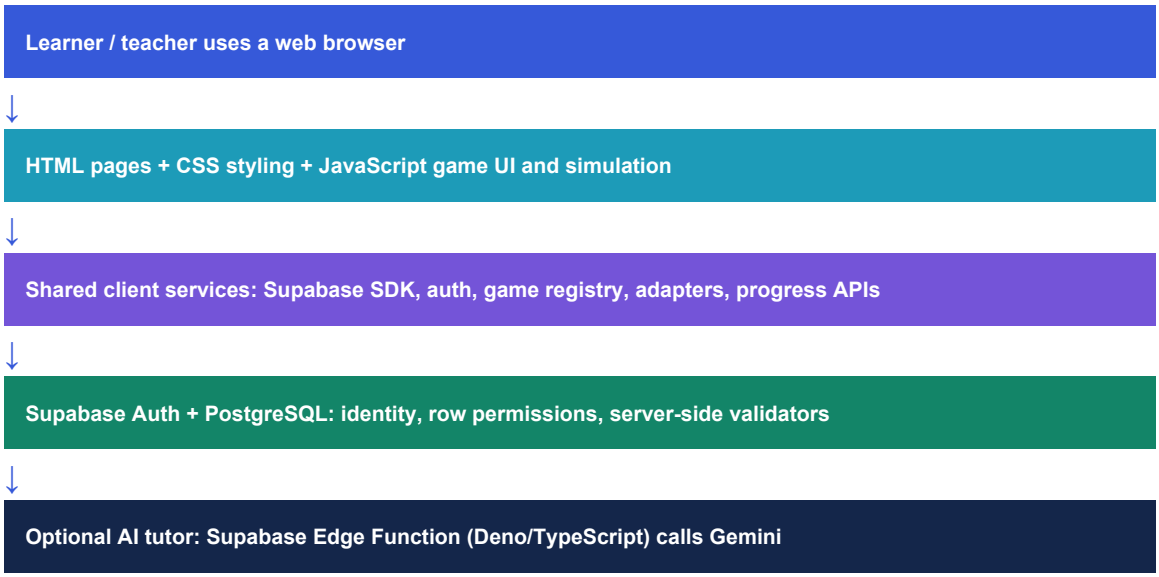
Like a school: the games are classrooms, the browser is the desk, and Supabase is the school office that checks the student badge and keeps the report card.

Prepared from the repository on branch master, 26 October 2026. This explains the checked-in code and configuration; it does not verify every live production setting.

1. What is this system?

RoboLogic STEM Lab is a web learning platform for middle-school learners. It teaches by letting students experiment: change a value, plan a path, build a molecule, or write a beginner code solution. The platform shows feedback and can save progress for a signed-in student.

Architecture map



Who does what?

Layer	Responsibility	Examples
Presentation	Draws the pages, controls, and feedback	HTML, CSS, dashboard.js, game-specific UI
Client game logic	Runs immediate simulation and packages an attempt	Physics trajectory, maze movement, submarine route
Shared platform	Gives different games the same login/save/reward interface	js/game-platform.js, game-auth.js, game-adapter.js
Backend	Checks identity, access, answers, progress, rewards	Supabase Auth, PostgreSQL, RLS, SQL RPCs
AI helper	Adds an optional short hint or question	Supabase analyze-mistake Edge Function + Gemini

Important distinction: this is mainly a browser app, not a custom Python or Node web server. Supabase provides the backend. The repository does not identify the production static-site host.

2. Technology stack, in plain words

A stack is the toolbox used to build the product.

Technology	Job	Where it appears
HTML5	Builds page structure	login.html, dashboard.html, game index pages
CSS3	Controls layout, colors, responsive design, motion	css/*.css and game styles
JavaScript / ES modules	Makes buttons, simulations, and game rules work	js/*.js and game folders
Supabase JavaScript SDK	Lets browser securely call Supabase services	vendor/supabase-browser.js and shared APIs
Supabase Auth	Creates accounts and manages signed-in sessions	Email/password sign-up, login, logout
PostgreSQL	Stores progress and trusted game data	Supabase database; migrations 001-036
RLS + SQL / PLpgSQL	Filters each student's rows and validates attempts	Policies and database RPC functions
Deno + TypeScript	Runs the server-side tutor endpoint	supabase/functions/analyze-mistake
Gemini API	Creates optional conversational hints	Called from the Edge Function; key stays server-side
Blockly 12	Lets learners build visual blocks	Chemistry Lab; vendor/blockly.min.js
CodeMirror	Editor component available to Chemistry	Chemistry/codemirror.bundle.js
Tailwind CSS output	Utility styles for Submarine Math	vendor/submarine-tailwind.css
Node.js + npm	Runs repo checks, tests, level generation	package.json; Node >=20
Playwright	Automates browser and accessibility checks	tests/browser
GitHub Actions	Runs automated checks on configured pushes/PRs	.github/workflows/quality.yml

Not shown: no React/Vue application, custom Express server, full Python runtime, or confirmed production hosting pipeline. The Locked Academy teaches Python, but its lesson runner is a small JavaScript evaluator for selected Python-like tasks.

3. How information travels

A data flow is a journey that information takes through the system. This is what happens when a learner completes a level.

1. OPEN: browser loads the page and game files

↓

2. SIGN IN: email/password go to Supabase Auth; Auth returns a signed session token

↓

3. LOAD: client requests this student's profile and progress

↓

4. PLAY: JavaScript runs a fast local simulation or editor

↓

5. SUBMIT: client sends game id, level, unique attempt id, and attempt data to an authenticated RPC

↓

6. CHECK: PostgreSQL validates the student, game release, level and game-specific answer

↓

7. SAVE: trusted logic records completion; reward logic awards stars/XP

↓

8. UPDATE: client refreshes the door, map, dashboard and feedback

Two sides of the system

Browser / client side	Supabase / server side
Shows screens and runs immediate interactions	Verifies the signed-in user and permissions
May do early format checks for helpful feedback	Applies RLS to rows and limits available operations
Sends a bounded attempt with the current session	Runs the authoritative game validator
Displays confirmed result	Writes progress, attempts and rewards

Think of the browser as the student handing in homework. The database is the teacher's gradebook. The browser can show a practice result quickly, but the database must confirm success before it becomes a saved reward.

AI hint flow: when supported, the browser sends limited mistake context to an authenticated Edge Function. The function checks the user, reserves quota, asks Gemini for a short hint, saves the insight, and returns it. If AI is unavailable, standard rule-based hints can still be shown.

4. Authentication, database, and security

Authentication means “who are you?” Authorization means “what are you allowed to do?” Both matter.

Piece	Like a five-year-old explanation	What code does
Supabase Auth	Student badge	Email/password sign-up, sign-in, sign-out and session handling
JWT session	A signed pass proving the badge was checked	Supabase client carries the session on backend calls
RLS	A name check on each drawer	Policies use auth.uid() so students see their own records
RPC validator	Teacher checks whether homework meets the rules	Authenticated SQL functions validate game attempts
Publishable/anon key	Public address for the app to call Supabase	Visible in browser by design; safe only with correct RLS and grants
Secret API key	Staff-only key	GEMINI_API_KEY must stay in Edge Function server environment
CSP	A guest list for what code/sites can load	Pages define Content-Security-Policy; script source is restricted to self
Idempotency	A receipt to stop a double-count	attempt_receipts saves a result by student + game + attempt id
AI quota	A limited number of tutor visits	Database quota reservation/refund and server-side checks

Database record-book tables

Table family	What it remembers
profiles	Student display name connected to Supabase Auth user id
games / game_levels	Enabled games, release count, level instructions/configuration
level_progress / level_attempts	Completion status, number of tries, result and timestamps
student_achievements	Stars and XP earned after success
mistake_events / ai_insights	Mistake category and tutor feedback
attempt_receipts	Processed submission ids and saved responses
*_answer_keys	Expected answers; direct client table access is revoked
tutor_quota_events	Tutor quota reservation/refund accounting
creator_levels / attempts / progress	RoboMaze creator drafts, published levels and play progress

Security reality: a browser is controlled by the user, so the server cannot trust browser success claims. Core migrations enable RLS, keep answer-key tables private, and restrict authoritative functions to authenticated users. Live security still depends on the deployed database having the right migrations, policies, grants, Auth settings, and secrets.

5. The five learning rooms

Game	Student activity	Concept and implementation
Physics Kinematics Lab	Adjust launch angle/speed; observe projectile	Trajectory and variables; canvas scene, JavaScript simulation, server validator
RoboLogic Maze	Program a robot through a grid	Sequencing, loops, conditions, debugging; grid simulation and program validator
Submarine Math Expedition	Plan signed depth moves through checkpoints/hazards	Integers and constraints; mission engine parses route and backend validates it
Chemistry Lab	Build target compounds and code challenges	Atoms/formulas/reactions; Blockly blocks, editor bundle and server validator
The Locked Academy	Find room clues, write beginner code, open doors	print(), strings, numbers, variables, arithmetic; constrained Python-like evaluator plus server validator

How the games connect

Each game has a stable technical id and a route. The registry says which games are enabled and how many levels are released. An adapter translates each game's result into one shared attempt format. The database boundary then dispatches to the matching game validator.

Locked Academy detail: it keeps the technical game id signal_decrypter so existing progress and reward contracts continue to work. The student sees the new name. Its lesson runner is intentionally limited and is not a full Python interpreter.

Learning loop and level unlock



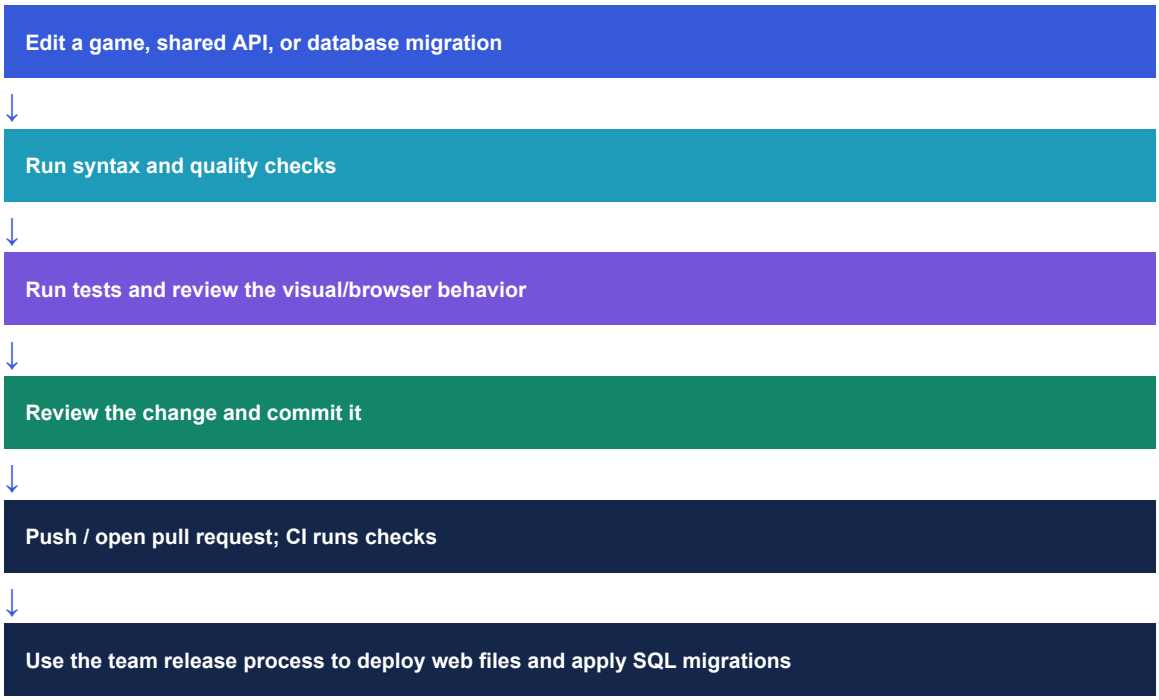
Why this approach matters

The UI can disable locked levels to guide the student, but JavaScript alone cannot secure them. The common database function also checks that the game is enabled and the submitted level is inside the released level count before calling a validator. Achievement logic separately checks the release limit.

6. How the team checks and changes the system

Tool	What it checks
npm run check	Repository rules, web quality, generated level data and level parity
npm test	Node unit/contract tests: APIs, simulation, persistence and game behavior
npm run test:browser	Playwright browser behavior and accessibility tests
npm run test:integration	Integration checks, including Supabase RPC behavior
GitHub Actions	Runs JS syntax, repo checks, tests, Deno Edge Function tests, file checks and a common secret-name scan
scripts/static-server.mjs	Serves pages locally during development

Normal change path



What we must not overclaim

Topic	Honest statement
Hosting	The repo contains static files/local server and CI checks, but does not prove the production static host or auto-deploy setup.
AI tutor	The code includes a Gemini Edge Function, quota and fallback. It works in production only if deployed and configured with secrets.
Learning impact	Progress records show usage/completions; they do not by themselves prove students learned more. That requires classroom testing and pre/post measurement.
School roles	Student accounts and creator features exist; a complete teacher/classroom administration system is not evident.
Student privacy	Progress and mistake data are stored, and some mistake context can be sent to AI. For school rollout, document consent, data retention, access and applicable child/student privacy rules.

A simple security slogan: Auth checks WHO; RLS checks WHICH ROWS; the validator checks WHETHER WORK COUNTS.

7. Presentation script and demo plan

Opening: “Many students meet STEM as facts on a page, but it can be hard to see how an idea behaves. RoboLogic STEM Lab turns those ideas into short challenges students can try, inspect, and retry. We built for middle-school learners who learn by doing.”

Problem: “Coding and STEM can feel abstract when students only read about them. A wrong answer may tell a student that something failed without helping them understand why. Students also need a clear next step and a way to see their progress.”

Solution: “Our web platform has interactive rooms for physics, robot logic, integer math, chemistry, and Python basics. Students experiment, get feedback, review mistakes, try again, and save their progress through one account.”

Demo route

Show	Say
Dashboard and sign-in	One student account keeps progress connected to the learner.
Open one game	Different rooms teach different STEM ideas using suitable interactions.
Try a deliberately wrong answer	The feedback makes the mistake visible and offers a next step.
Review and retry	This is the learning loop: attempt, feedback, reflection, improvement.
Complete a level	Show success, saved progress, stars, and the next unlocked level. The database confirms the result.

Technical explanation: “The front end is HTML, CSS, and JavaScript. Supabase Auth manages accounts; PostgreSQL stores progress. Row-level security scopes records to each user. Game submissions go to server-side database functions, which validate an attempt before it earns saved progress or rewards. An optional Supabase Edge Function calls Gemini to create a hint; standard hints are a fallback.”

Difference: “Our focus is hands-on experimentation and revision. The learner changes something, observes the result, and uses feedback to improve. Multiple STEM activities share one account and progress system, and server validation protects saved achievements.”

Closing: “We help learners move from seeing a STEM idea to trying it, explaining what happened, and improving their next attempt. Our next step is to test with middle-school students and use their feedback to improve clarity and accessibility.”

8. Judge Q&A and mini glossary

Question	Answer
How do you know they learned?	The app records attempts and completion, but records alone do not prove learning. We will test with students and compare understanding before/after.
What is your strongest technical feature?	A reusable game platform and common authenticated submission path that routes games to trusted game-specific validators.
What stops self-awarded stars?	The browser does not decide permanent success. Authenticated database functions validate attempts and enforce release limits before saving rewards.
Why multiple games?	One platform shares account/progress, while each concept gets an interaction that fits it: trajectory, route planning, integer movement, molecule assembly, or code.
What is a limitation?	We need measured student learning evidence, classroom testing, accessibility review, and confirmation of production configuration and privacy requirements.
Why AI?	A short hint/question can help students think. The tutor should not give away the full solution, and rule-based fallback hints remain available.

Glossary

Word	Simple meaning
API	A doorway one program uses to request something from another.
RPC	A request to run a named function on the database server.
JWT	A signed digital pass showing who has authenticated.
RLS	Database rules for which rows each user may read or change.
Migration	A numbered SQL change to the database structure or rules.
Validator	A trusted checker for whether an answer follows the level's rules.
Idempotent	Safe to repeat; a retry does not apply the same result twice.
Edge Function	A small server-side program hosted by Supabase.
Client/server	The browser the learner uses / trusted backend and database.

Main files to open when presenting the technical design

package.json; workflow quality.yml; js/game-auth.js; game-platform.js; game-adapter.js; progress-api.js.
Database: supabase/migrations/001 and 020-036.
AI: supabase/functions/analyze-mistake/handler.ts. Games: PhysicsSim, RoboMaze, SubmarineMath, Chemistry, SignalDecrypter

The repository proves what code is present, not what is deployed or measured. Verify live migrations, secrets, hosting, and learning outcomes before making claims about them.